Geometry Note

Ran Xie

August 20, 2026

Modern Terrain Generation & Rendering

A compact mathematical guide for real-time game engines

Philosophy

A modern terrain system can be decomposed into four largely independent problems:

$$\boxed{\begin{array}{c}\text{Terrain}\\ \hline \text{Shape} + \text{Materials} + \text{Representation} + \text{Visibility}\end{array}}$$

where

$$\text{Shape} = H(\mathbf{x})$$

defines the world,

$$\text{Materials} = M(\mathbf{x})$$

defines its appearance,

$$\text{Representation}$$

decides how much geometry/data exists at each scale, and

$$\text{Visibility}$$

decides what the GPU actually processes.

The key conceptual shift from older terrain programming is:

$$\boxed{\text{Do not think “generate a mesh”.}}$$

Think:

$$\boxed{\text{Define a continuous terrain function and sample it adaptively.}}$$

GPU terrain systems have increasingly moved toward this separation: geometry clipmaps use nested regular grids, while modern mesh-shader approaches can generate terrain patches directly on the GPU.

Part I — Terrain as a mathematical field

0.1 Heightfield

For ordinary terrain:

$$H : \mathbb{R}^2 \to \mathbb{R}$$

and

$$\boxed{\mathbf{P}(x,z) = (x, H(x,z), z)}$$

is the terrain surface.

Everything else can be derived from H.

Height:

$$H(x,z)$$

slope:

$$\nabla H = \begin{pmatrix} H_x \\ H_z \end{pmatrix}$$

normal:

$$\boxed{\mathbf{N} = \frac{(-H_x, 1, -H_z)}{\sqrt{1 + H_x^2 + H_z^2}}}$$

curvature depends on the second derivatives:

$$H_{xx},\ H_{xz},\ H_{zz}.$$

This is important because terrain generation does not need to directly produce vertices. It can instead produce

$$H(x,z)$$

and the renderer samples it.

0.2 Why simple noise looks bad

A common terrain generator is

$$H(x,z) = N(x,z).$$

This produces something that looks like random variation.

Instead use multiple spatial frequencies:

$$\boxed{H(\mathbf{x}) = \sum_{i=0}^{n} A_i N(f_i \mathbf{x})}$$

with

$$f_{i+1} = 2 f_i$$

and

$$A_{i+1} = r A_i, \qquad 0 < r < 1.$$

Therefore:

$$H = A_0 N(\mathbf{x}) + r A_0 N(2\mathbf{x}) + r^2 A_0 N(4\mathbf{x}) + \cdots$$

This is fractional Brownian motion (fBm)-style terrain.

A typical parameterization is

$$A_i = f_i^{-H_f}$$

where H_f controls roughness.

Large A_i:

$$\Rightarrow \text{large mountains}$$

small A_i:

$$\Rightarrow \text{fine detail.}$$

0.3 Ridged terrain

Ordinary noise has positive and negative regions. For mountain ridges, transform it:

$$r(\mathbf{x}) = 1 - |N(\mathbf{x})|.$$

Then optionally square:

$$\boxed{R(\mathbf{x}) = (1 - |N(\mathbf{x})|)^2}$$

which makes high values concentrate around the zero crossings of the original noise.

A fractal ridged terrain becomes

$$H(\mathbf{x}) = \sum_i A_i R(f_i \mathbf{x}).$$

0.4 Domain warping

One of the most useful upgrades over basic noise is to distort the input domain.

Let

$$\mathbf{D}(\mathbf{x}) = \begin{pmatrix} N_x(\mathbf{x}) \\ N_z(\mathbf{x}) \end{pmatrix}.$$

Warp the coordinate:

$$\boxed{\mathbf{x}' = \mathbf{x} + \lambda \mathbf{D}(\mathbf{x})}$$

and evaluate

$$\boxed{H(\mathbf{x}) = N(\mathbf{x}').}$$

Multiple warps can be nested:

$$\mathbf{x}_1 = \mathbf{x} + \lambda_1 D_1(\mathbf{x})$$
$$\mathbf{x}_2 = \mathbf{x}_1 + \lambda_2 D_2(\mathbf{x}_1)$$
$$H = N(\mathbf{x}_2).$$

This produces structures much less obviously related to the underlying noise grid.

0.5 Separate terrain scales

A useful terrain model is:

$$\boxed{H = H_{\text{macro}} + H_{\text{meso}} + H_{\text{micro}}}$$

where

$$H_{\text{macro}} = N_{\text{low}}(\mathbf{x})$$

defines continents/mountain ranges,

$$H_{\text{meso}} = \sum_i A_i N_i(f_i \mathbf{x})$$

defines hills and valleys, and H_{micro} contains small-scale detail.

This separation is extremely useful because different scales can have different generation rules.

For example:

$$H_{\text{macro}} \to \text{mountain ranges}$$
$$H_{\text{meso}} \to \text{ridges/valleys}$$
$$H_{\text{micro}} \to \text{rocks/roughness}.$$

0.6 From noise to geological structure

A better conceptual model is:

$$\boxed{H = H_{\text{tectonic}} + H_{\text{erosion}} + H_{\text{deposition}} + H_{\text{detail}}.}$$

Noise is only one component.

For example, suppose erosion transforms terrain according to

$$H_{t+\Delta t} = H_t + \Delta H_{\text{erosion}}.$$

Then

$$\boxed{H_{n+1} = E(H_n)}$$

where E is an erosion operator.

Repeated application:

$$H_{\text{final}} = E^n(H_0).$$

This is fundamentally different from simply adding more noise.

Part II — Erosion

0.7 Hydraulic erosion

Represent water as a field

$$W(x, z, t).$$

Represent sediment as

$$S(x, z, t).$$

Water moves according to terrain gradients:

$$\mathbf{v} \propto -\nabla(H + W).$$

Sediment transport can be approximated as

$$\frac{\partial S}{\partial t} + \nabla \cdot (S\mathbf{v}) = D_{\text{erosion}} - D_{\text{deposition}}.$$

A simple erosion/deposition model is

$$\Delta H = -E_{\text{capacity}} + D_{\text{deposition}}.$$

The important idea is:

$$\boxed{\text{erosion follows flow, not random noise.}}$$

Consequently:

$$\text{mountain} \rightarrow \text{water flow} \rightarrow \text{valley} \rightarrow \text{sediment}.$$

0.8 Thermal erosion

If a slope becomes too steep, material falls downhill.

Let

$$s = |\nabla H|.$$

Define a maximum stable slope

$$s_{\max}.$$

Then material moves whenever

$$\boxed{s > s_{\max}.}$$

A simple transfer is

$$\Delta H_{\text{source}} = -k(s - s_{\max})$$

and

$$\Delta H_{\text{neighbor}} = +\frac{k(s - s_{\max})}{n}.$$

Thermal erosion therefore tends to create:

$$\text{cliffs} \rightarrow \text{talus} \rightarrow \text{stable slopes}.$$

Part III — Biomes and procedural materials

0.9 Don't classify terrain using height alone

Instead construct a feature vector:

$$\boxed{\mathbf{F} = (H, |\nabla H|, \kappa, T, W, D_{\text{water}}, \ldots)}$$

where:

- H = altitude
- $|\nabla H|$ = slope
- κ = curvature
- T = temperature
- W = moisture
- D_{water} = distance to water.

Then define biome weights:

$$\boxed{w_i = f_i(\mathbf{F}).}$$

Normalize:

$$\boxed{\hat{w}_i = \frac{w_i}{\sum_j w_j}.}$$

This gives

$$\sum_i \hat{w}_i = 1.$$

0.10 Temperature

A simple model:

$$T = T_0 - \alpha H.$$

Thus altitude naturally decreases temperature.

Add latitude:

$$T = T_0 - \alpha H - \beta|\phi|.$$

Noise can provide local variation:

$$T = T_0 - \alpha H - \beta|\phi| + \gamma N_T(\mathbf{x}).$$

0.11 Moisture

Moisture can be represented as another scalar field:

$$M(\mathbf{x}).$$

For a simple model:

$$M = M_0 + N_M(\mathbf{x}) - \lambda D_{\text{water}}.$$

Then biome classification becomes a function of

$$(T, M).$$

For example:

$$(T, M) \to \begin{cases} \text{desert} \\ \text{grassland} \\ \text{forest} \\ \text{tundra} \\ \text{snow}. \end{cases}$$

This is essentially a continuous biome map rather than a collection of hard-coded regions.

0.12 Terrain material blending

Let the terrain contain materials

$$T_1, T_2, \ldots, T_n.$$

The final material is

$$\boxed{C = \sum_i w_i T_i.}$$

with

$$\sum_i w_i = 1.$$

For example:

$$w_{\text{rock}} = f(\text{slope})$$
$$w_{\text{snow}} = f(H, T)$$
$$w_{\text{grass}} = f(H, T, M).$$

The terrain shader therefore becomes a **material field evaluator**.

0.13 Triplanar texturing

For arbitrary terrain, planar UVs cause stretching.

Project the texture along three axes:

$$T_x(\mathbf{p}), \qquad T_y(\mathbf{p}), \qquad T_z(\mathbf{p}).$$

Use the normal to determine the blending weights:

$$w_x = |N_x|^p, \quad w_y = |N_y|^p, \quad w_z = |N_z|^p.$$

Normalize:

$$\boxed{\hat{w}_i = \frac{w_i}{w_x + w_y + w_z}.}$$

Then

$$\boxed{C = \hat{w}_x T_x + \hat{w}_y T_y + \hat{w}_z T_z.}$$

This technique is also used for arbitrary-topology procedural terrain; GPU Gems 3 presents triplanar mapping specifically as a solution to planar-projection distortion.

0.14 Slope-aware material

A useful rock mask:

$$s = 1 - N_y.$$

Then

$$w_{\text{rock}} = \text{smoothstep}(s_0, s_1, s).$$

For snow:

$$w_{\text{snow}} = \text{smoothstep}(H_0, H_1, H) \cdot (1 - w_{\text{rock}}).$$

The result:

$$\boxed{C = w_r T_r + w_g T_g + w_s T_s.}$$

Part IV — Rendering the terrain

0.15 The old model

The naive renderer is:

$$H_{ij} \to \mathbf{P}_{ij} \to \text{triangle mesh}.$$

For a grid of $N \times N$:

$$O(N^2)$$

vertices.

This becomes impossible for sufficiently large worlds.

The solution is not necessarily to make the mesh itself more clever. Instead:

sample the terrain at the resolution actually required by the camera.

0.16 Screen-space error

Suppose a terrain approximation has world-space error

$$E_w.$$

At camera distance d, its approximate projected error is

$$\boxed{E_s \approx \frac{fE_w}{d}}$$

where f is the camera projection scale.

Choose a tolerance

$$\epsilon.$$

Then refine whenever

$$\boxed{E_s > \epsilon.}$$

This equation is the conceptual heart of modern terrain LOD. Distance alone is merely an approximation to this.

0.17 Quadtree

Divide terrain recursively:

$$Q \to \{Q_{00}, Q_{01}, Q_{10}, Q_{11}\}.$$

At depth k:

$$\text{world size} \propto 2^{-k}.$$

For each node calculate:

$$E(Q).$$

Then:

$$\boxed{E_s(Q) > \epsilon \Rightarrow \text{split.}}$$

Otherwise:

$$\boxed{E_s(Q) \leq \epsilon \Rightarrow \text{render node.}}$$

This gives spatially adaptive terrain resolution.

0.18 Geometry clipmaps

Geometry clipmaps take a different approach.

Maintain nested grids:

$$G_0, G_1, \ldots, G_L.$$

Their spatial resolutions satisfy approximately

$$\boxed{\Delta_{i+1} = 2\Delta_i.}$$

The finest grid covers a small region around the camera. The next grid covers a larger region. And so on.

Each level is essentially a ring:

$$G_i = \text{outer square} - \text{inner square}.$$

The inner square is already covered by

$$G_{i-1}.$$

Geometry clipmaps therefore achieve large-world coverage using a small number of fixed-size grids. The original GPU implementation stores elevation as textures and shifts the nested windows as the viewer moves.

0.19 Clipmap transition

At the boundary between levels, directly switching resolution causes a crack/popping artifact.

Define

$$\alpha = \text{smoothstep}(0, 1, t)$$

where t is distance through the transition region.

Blend heights:

$$\boxed{H_{\text{final}} = (1 - \alpha)H_{\text{fine}} + \alpha H_{\text{coarse}}.}$$

The same idea can be applied to normals and other data.

0.20 Virtual terrain data

For a truly large world, don't store everything at maximum resolution.

Define a hierarchy:

$$D_0, D_1, \ldots, D_L.$$

where

$$D_{i+1} = \text{downsample}(D_i).$$

At runtime:

$$\boxed{\text{resident data} = \text{only what the camera needs.}}$$

This principle applies to both geometry and textures.

0.21 GPU-driven terrain

The modern rendering model increasingly moves visibility and geometry decisions toward the GPU.

Conceptually:

$$\text{terrain patches} \rightarrow \text{GPU culling} \rightarrow \text{visible patches} \rightarrow \text{mesh generation} \rightarrow \text{rasterization}.$$

Instead of the CPU deciding every patch:

$$CPU \rightarrow \{D_1, D_2, D_3, \ldots\}$$

we want:

$$\boxed{GPU \rightarrow \text{visible patch list.}}$$

Modern APIs explicitly support this style: mesh shaders process small groups of geometry cooperatively, while amplification shaders can cull or launch mesh-shader work.

0.22 Terrain with mesh shaders

A terrain patch can be generated procedurally.

For a patch coordinate

$$(i, j)$$

generate grid vertices:

$$u = \frac{i}{N}, \qquad v = \frac{j}{N}.$$

Map to world position:

$$x = x_0 + uL, \quad z = z_0 + vL.$$

Evaluate:

$$\boxed{y = H(x, z).}$$

Thus:

$$\boxed{P_{ij} = (x_0 + uL,\ H(x_0 + uL, z_0 + vL),\ z_0 + vL).}$$

No persistent mesh for the patch is strictly necessary. A modern GPU can generate the patch geometry directly.

AMD's current mesh-shader material explicitly demonstrates procedural terrain this way: mesh-shader groups generate regular terrain sections while changing vertex spacing between LODs.

0.23 Mesh-shader LOD

Suppose a patch has physical size L. LOD k uses spacing

$$\boxed{\Delta_k = 2^k \Delta_0.}$$

Therefore the number of vertices decreases approximately as

$$N_k \propto \frac{L^2}{\Delta_k^2} = \frac{L^2}{4^k \Delta_0^2}.$$

So every increase of one LOD level reduces the 2D vertex density by approximately

$$\boxed{4\times}.$$

This is the mathematical reason hierarchical LOD works so well.

0.24 A useful hybrid architecture

For a modern game, a very practical abstraction is:

$$\boxed{\text{Quadtree} + \text{GPU culling} + \text{procedural patch generation.}}$$

The quadtree answers:

"where should resolution change?"

GPU culling answers:

"which nodes matter?"

and the mesh shader answers:

"generate the triangles."

This separates the three problems cleanly.

Part V — Terrain continuity

0.25 The crack problem

Suppose adjacent patches use resolutions

$$\Delta_A$$

and

$$\Delta_B = 2\Delta_A.$$

Their vertices don't necessarily line up. This produces T-junctions/cracks.

There are several solutions.

Stitching Add triangles connecting the two resolutions.

Skirts Extend patch boundaries downward.

Morphing Move fine vertices toward the coarse representation.

The clean mathematical solution is:

$$\boxed{P' = (1-\alpha)P_{\text{fine}} + \alpha P_{\text{coarse}}.}$$

0.26 Geometric morphing

Let

$$H_f$$

be fine height and

$$H_c$$

the corresponding coarse approximation. Then

$$\boxed{H' = H_f + \alpha(H_c - H_f).}$$

At

$$\alpha = 0: \quad H' = H_f.$$

At

$$\alpha = 1: \quad H' = H_c.$$

This removes visible popping when switching LOD.

Part VI — Terrain texturing at enormous scale

0.27 Why ordinary UV textures fail

Suppose terrain covers

$$100\,\text{km}^2.$$

A single texture with texel density r requires approximately

$$N \approx Lr$$

texels per dimension. Storage scales as

$$\boxed{O(L^2 r^2).}$$

Eventually it becomes absurd.

0.28 Texture hierarchy

Instead use tiled mip levels:

$$T_0, T_1, \ldots, T_L.$$

Each level has approximately

$$\frac{1}{2}$$

the resolution of the previous level. Thus:

$$T_{i+1} = \text{downsample}(T_i).$$

The renderer only needs a small neighborhood of the hierarchy.

0.29 Virtual texturing

Conceptually:

$$\boxed{\text{world texture} \rightarrow \text{tiles} \rightarrow \text{resident tile cache.}}$$

The shader requests texels. A feedback mechanism records:

$$F(x) = \text{which texture region/mip was sampled.}$$

The streaming system then loads missing tiles.

Modern GPU APIs explicitly expose sampler feedback for recording which texture regions and mip levels were accessed, enabling this kind of runtime texture streaming.

0.30 Procedural texturing can avoid storage entirely

If the material is procedural:

$$C = C(H, \nabla H, N, T, M, \mathbf{x})$$

then there is no requirement to store a giant albedo texture.

For example:

$$C = w_g T_g(\mathbf{x}) + w_r T_r(\mathbf{x}) + w_s T_s(\mathbf{x}).$$

The terrain texture is effectively a function:

$$\boxed{T : \mathbb{R}^3 \rightarrow \mathbb{R}^4.}$$

This is especially attractive for stylized/NPR terrain.

Part VII — Advanced terrain representation

0.31 Heightfields have a fundamental limitation

A heightfield requires:

$$(x, z) \rightarrow y.$$

Therefore for every (x, z) there can only be one height. It cannot naturally represent:

- caves
- arches
- overhangs

- tunnels
- cliffs with overlapping surfaces.

For these, define a 3D scalar field:

$$\boxed{D(x, y, z)}$$

and define the surface implicitly:

$$\boxed{D(x, y, z) = 0.}$$

Terrain becomes an **implicit surface**.

0.32 Signed-distance / density terrain

A signed distance field approximately satisfies

$$|D(\mathbf{x})| = \text{distance to surface.}$$

Then:

$$D < 0$$

can represent inside,

$$D > 0$$

outside, and

$$D = 0$$

the terrain surface.

A terrain can therefore be generated as:

$$\boxed{D(\mathbf{x}) = D_{\text{mountain}}(\mathbf{x}) + D_{\text{cave}}(\mathbf{x}) + D_{\text{rock}}(\mathbf{x}).}$$

The surface is extracted afterward.

GPU Gems 3 demonstrates this broader procedural-terrain idea using a 3D density function and marching cubes; it specifically uses the density field to permit features such as caves and overhangs that ordinary heightfields cannot represent.

0.33 Marching cubes

Given a scalar field

$$D(x, y, z),$$

sample it on a voxel grid.

For every voxel:

$$D_0, \ldots, D_7$$

determine which corners are inside/outside.

The zero surface intersects edges where

$$D_a D_b < 0.$$

Linear interpolation gives the intersection:

$$\boxed{t = \frac{D_a}{D_a - D_b}}$$

and

$$\boxed{P = (1 - t)P_a + tP_b.}$$

The resulting polygonization creates an arbitrary-topology terrain mesh.

Part VIII — Lighting

0.34 Terrain normal

For heightfields:

$$\boxed{N = \frac{(-H_x, 1, -H_z)}{\sqrt{1 + H_x^2 + H_z^2}}.}$$

For discrete samples:

$$H_x \approx \frac{H(x+\Delta, z) - H(x-\Delta, z)}{2\Delta}$$
$$H_z \approx \frac{H(x, z+\Delta) - H(x, z-\Delta)}{2\Delta}.$$

0.35 Diffuse lighting

For directional light L:

$$\boxed{I_d = \max(0, N \cdot L).}$$

Add ambient:

$$\boxed{I = A + D\max(0, N \cdot L).}$$

Then:

$$C_{\text{final}} = C_{\text{material}} I.$$

For NPR rendering, however, there is no requirement to preserve the physical Lambertian function. You can replace it with:

$$I = f(N \cdot L)$$

where f is an arbitrary artistic transfer function.

For example:

$$f(x) = \begin{cases} 0.4, & x < 0.2 \\ 0.7, & 0.2 \le x < 0.6 \\ 1, & x \ge 0.6. \end{cases}$$

This is often more useful for stylized terrain than physically based lighting.

0.36 Curvature

Second derivatives provide useful terrain information.

Approximate curvature indicators using:

$$H_{xx} \approx \frac{H(x+\Delta, z) - 2H(x, z) + H(x-\Delta, z)}{\Delta^2}$$

and

$$H_{zz} \approx \frac{H(x, z+\Delta) - 2H(x, z) + H(x, z-\Delta)}{\Delta^2}.$$

Then a simple curvature field is

$$\boxed{K \approx H_{xx} + H_{zz}.}$$

This can drive:

- rock placement
- erosion
- snow accumulation
- vegetation
- edge highlighting
- NPR shading.

Part IX — The modern architecture

0.37 CPU/GPU responsibilities

A clean architecture is:

CPU

$$\boxed{\text{world management} + \text{streaming} + \text{high-level LOD}}$$

GPU

$$\boxed{\text{culling} + \text{terrain sampling} + \text{mesh generation} + \text{shading}}$$

The CPU should not continually construct thousands of tiny terrain meshes.

Modern GPU-driven approaches increasingly move geometry selection and generation onto the GPU. Microsoft's mesh-shader model explicitly replaces much of the fixed geometry pipeline with compute-like group processing, while newer work-graph systems push GPU autonomy further.

0.38 A practical terrain pipeline

A modern terrain frame can conceptually be:

1. Camera
2. Select terrain nodes
3. Frustum/occlusion cull
4. Determine LOD
5. Generate visible patches
6. Sample $H(x, z)$
7. Calculate normals/features
8. Evaluate material
9. Shade
10. Rasterize

The critical point is that steps 2–8 do not necessarily need to happen on the CPU.

Part X — A terrain system I'd actually build

For a modern game, I would start with:

$$\boxed{H(x,z) = H_{\text{macro}} + H_{\text{ridge}} + H_{\text{detail}}}$$

then:

$$\boxed{\mathbf{F} = (H, \nabla H, \nabla^2 H, T, M)}$$

for terrain classification.

Then:

$$\boxed{\text{Quadtree} \rightarrow \text{LOD selection} \rightarrow \text{GPU patch generation.}}$$

For each visible patch:

$$P_{ij} = (x_i, H(x_i, z_j), z_j).$$

For large worlds:

$$\boxed{\text{streamed height/material tiles}}$$

or procedural evaluation.

For rendering:

$$\boxed{\text{mesh shaders / GPU-driven draws.}}$$

For materials:

$$\boxed{C = \sum_i w_i(H, \nabla H, \kappa, T, M) T_i.}$$

For NPR:

$$\boxed{C_{\text{final}} = g(C, N, L, \text{view}, \text{style})}$$

rather than assuming physically based shading.

Part XI — What to learn, in order

If the objective is actually building the system, I'd study these in this order:

0.39 Scalar fields

Understand:

$$H(x,z), \quad \nabla H, \quad \nabla^2 H.$$

This gives you the mathematical language of terrain.

0.40 Noise / fractals

Understand:

$$\sum_i A_i N(f_i \mathbf{x})$$

and domain warping.

0.41 Erosion

Understand:

$$\text{height} \to \text{water flow} \to \text{sediment} \to \text{modified height}.$$

0.42 Material fields

Understand:

$$(H, \nabla H, T, M, \kappa) \to (w_1, \ldots, w_n).$$

0.43 Hierarchical LOD

Understand:

$$E_{\text{screen}} \approx \frac{fE}{d}.$$

0.44 Quadtree / clipmap

Understand how to represent an effectively enormous terrain with a bounded amount of active geometry.

0.45 GPU-driven rendering

Understand:

$$\text{candidate patches} \to \text{GPU culling} \to \text{visible patches}.$$

0.46 Mesh shaders

Understand:

$$\text{visible patch} \to \text{procedurally generated meshlet}.$$

0.47 Virtual texturing

Understand:

$$\text{world texture} \to \text{tile hierarchy} \to \text{resident cache}.$$

0.48 Only then optimize

Because the fundamental optimization is not:

$$\text{“make this terrain mesh faster”}$$

but:

$$\boxed{\text{“don’t generate/process terrain that nobody can see.”}}$$

Final mental model

The entire subject can ultimately be reduced to this:

$$\boxed{H(\mathbf{x})}$$

defines the terrain.

$$\boxed{F(\mathbf{x})}$$

defines its geological/material properties.

$$\boxed{E(\mathbf{x})}$$

defines how accurately it needs to be represented.

Then:

$$\boxed{\text{camera} \to E \to \text{LOD} \to \text{visibility} \to \text{sampling} \to \text{GPU geometry.}}$$

And finally:

$$\boxed{(H, F, N, L, \text{view}) \to C_{\text{final}}.}$$

That is the modern terrain renderer in a nutshell.

The three papers/chapters worth reading alongside this

Geometry Clipmaps — GPU Gems 2, Chapter 2. Still worth reading despite its age because the mathematical/data-structure idea remains excellent: nested regular grids, incremental updates, transition regions, and GPU-side terrain processing.

Generating Complex Procedural Terrains Using the GPU — GPU Gems 3, Chapter 1. Read it mainly for the conceptual jump from heightfields to density fields, GPU procedural generation, triplanar texturing, and variable-size terrain blocks.

Modern mesh-shader terrain / procedural generation. For the modern end of the pipeline, AMD's procedural-generation example is particularly relevant: it generates 8×8 terrain sections in mesh-shader groups and varies vertex spacing for LOD rather than constructing conventional CPU-side meshes.

The important distinction is that the first two are **foundations**, not necessarily the architecture I'd copy today. Mesh shaders and GPU-driven rendering change the implementation model substantially.